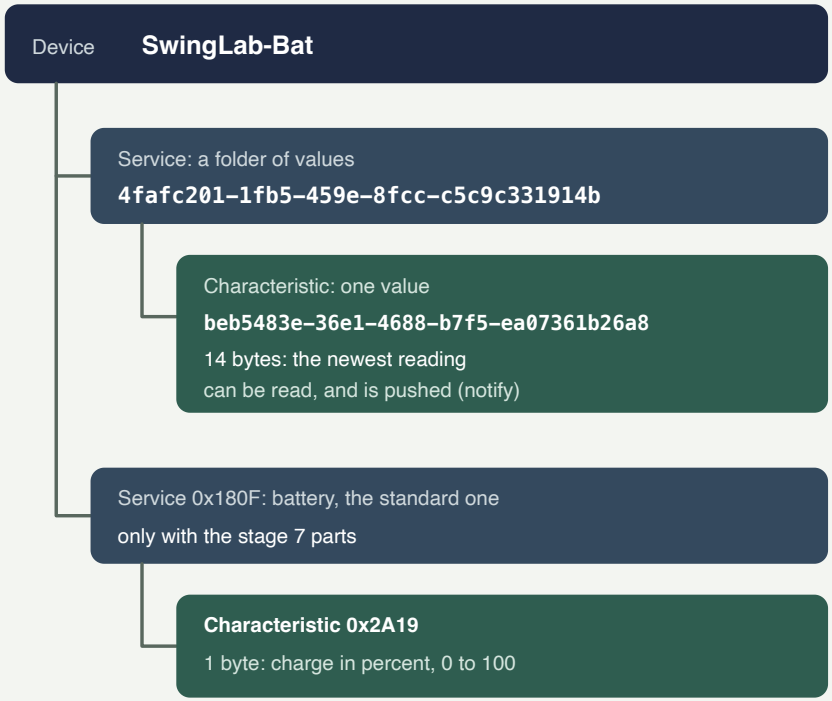
How the bat and the dashboard find each other over Bluetooth

Bluetooth Low Energy. The bat is the peripheral: it holds the data. The browser is the central: it connects and listens.

What the bat offers

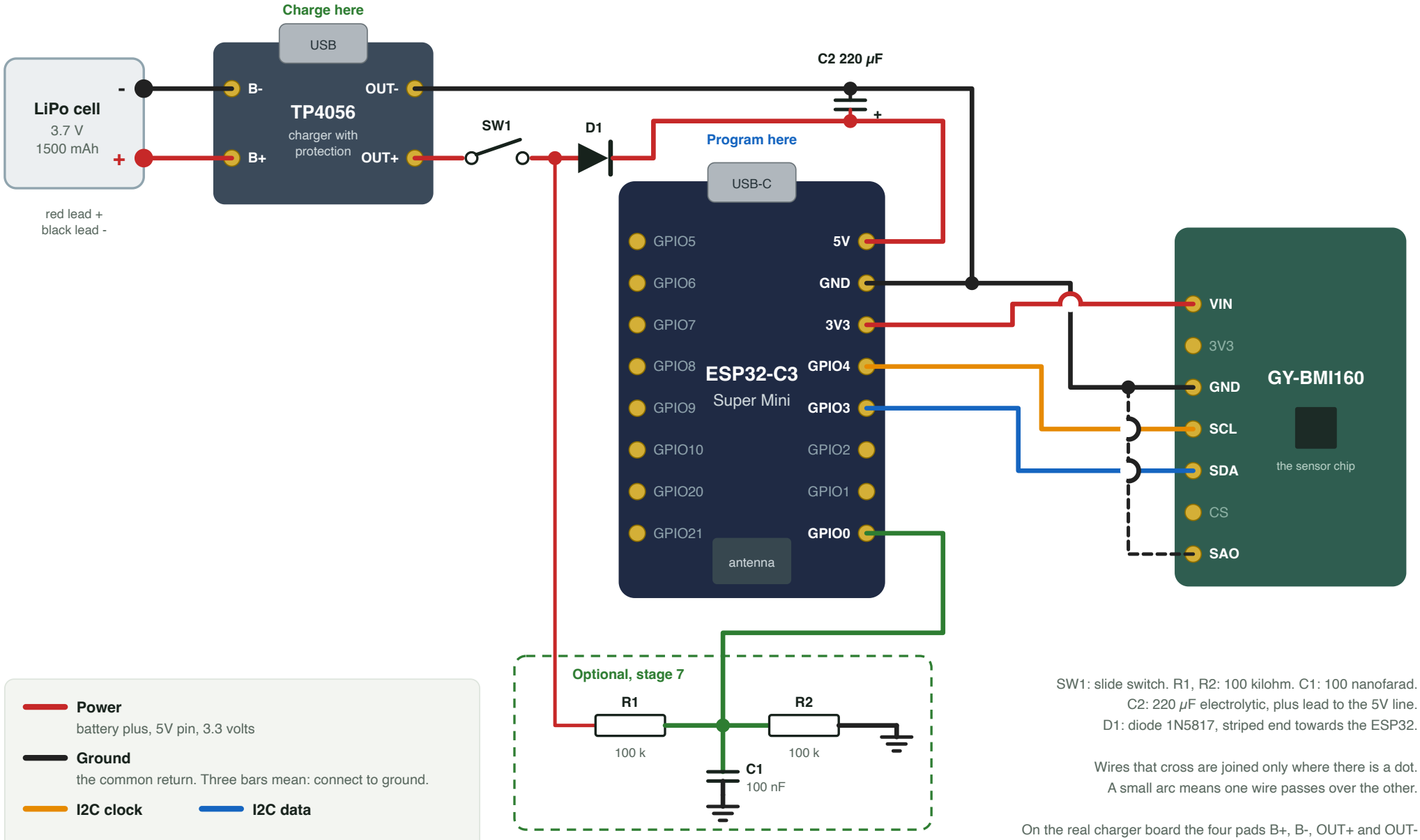


The long ids were picked at random, once. They only have to be the same in the firmware (config.h) and in the web app (sensor-packet.ts).

1. **The bat advertises**
   A few times a second it broadcasts: my name is SwingLab-Bat, and I offer service 4fafc201. Its light blinks slowly.
2. **You press Connect Sensor**
   The browser lists nearby devices whose name starts with SwingLab. A web page may only see what you pick yourself.
3. **The browser connects**
   From now on the two take turns many times a second. The bat stops advertising and its light stays on.
4. **The page finds the value**
   It asks for the service by its id, then for the characteristic by its id.
5. **The page asks for notifications**
   It flips a small switch on the characteristic that means: push me every new value.
6. **The bat sends 14 bytes every 20 ms**
   Nobody has to ask again. The dashboard uses a packet only while a shot is being recorded.
7. **The connection ends**
   The page closes or goes out of range. The bat starts advertising again by itself.

The whole circuit

Everything stage 8 needs. The part inside the dashed frame is optional and belongs to stage 7.



SW1: slide switch. R1, R2: 100 kilohm. C1: 100 nanofarad.
C2: 220 µF electrolytic, plus lead to the 5V line.
D1: diode 1N5817, striped end towards the ESP32.

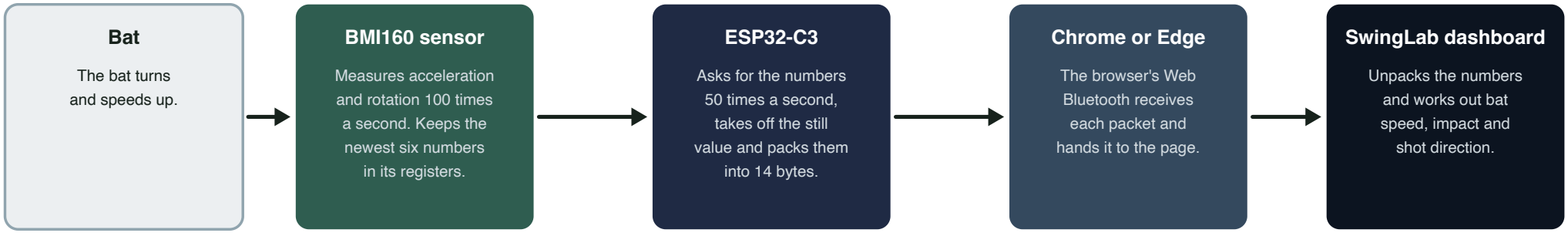
Wires that cross are joined only where there is a dot.
A small arc means one wire passes over the other.

On the real charger board the four pads B+, B-, OUT+ and OUT- sit side by side at the end away from its USB socket.

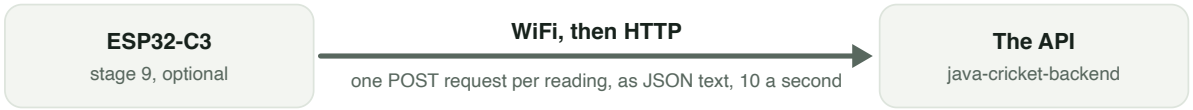
Dashed black line: SAO to GND, which fixes the sensor's address.

From a swing to a number on the screen

What each part does, and what travels between them.



The other road in:
the same readings can go straight to the backend, without a browser. The dashboard does not need this.



One reading on the I2C wires

What passes along the SDA wire when the ESP32 fetches the 12 data bytes. Time runs from left to right.



Two wires, shared by every device. SCL is the clock: the ESP32 ticks it, and one bit of data moves on SDA with every tick. Nobody talks unless the ESP32 asks.

Address. Every device has a number. This sensor is 0x68 when its SAO pin is joined to GND and 0x69 when SAO is joined to 3.3 volts. Only the device called answers.

Register. Inside the sensor the values sit in numbered boxes. 0x0C is the first of twelve boxes holding the newest reading. After each byte the sensor moves on to the next box by itself, which is why one request brings all twelve.

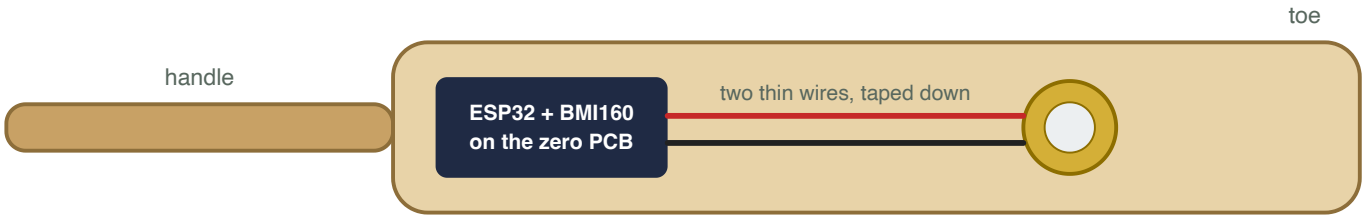
ACK. After every byte the receiver pulls SDA low for one tick to say "got it". No ACK after the address means nobody is there: that is what stage 2 looks for.

Speed. Stage 2 and 3 use 100 000 ticks a second. Later stages use 400 000, and the whole exchange above then takes about a third of a millisecond.

Pull-up resistors hold both wires at 3.3 volts when nobody pulls them low. The GY-BMI160 board has them built in, so none are added in this kit.

Impact sensor, a proposal: a piezo disc, no diodes needed

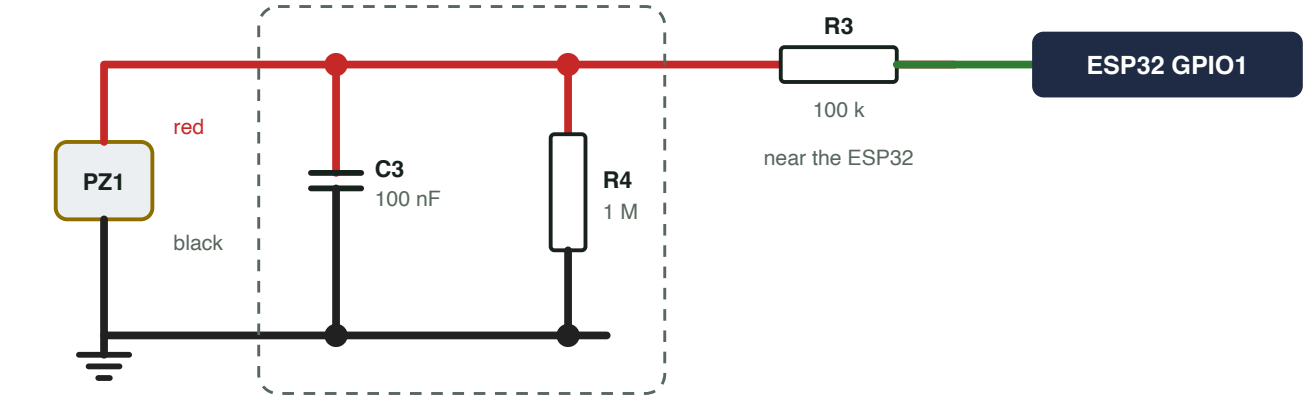
The firmware does not read it yet. It works the same with or without the diode in the power path.



PZ1 on the back of the blade, at the middle

roughly 15 to 20 cm from the toe

The circuit



Why no diode is needed

A hard hit can make the disc give tens of volts. C3 sits across it and takes most of that charge: the voltage falls to about a sixth.

R3 then lets only a fraction of a milliamp into the pin, which the pin's own protection inside the chip takes.

R4 empties the disc after each hit. The reading is back at zero in about a tenth of a second.

GPIO1 is free: GPIO0 measures the battery, GPIO3 and GPIO4 talk to the BMI160.

On the zero PCB

R3's free end in J8, joined to I8 (GPIO1) with a blob of solder.
The black wire in F14, joined to G14 (GND) the same way.
Both holes are free in both layouts.

PZ1: plain piezo disc, 27 mm, with two wires. Not a buzzer module. C3: 100 nF ceramic, printed 104. R4: 1 megohm (brown, black, green).

R3: 100 kilohm (brown, black, yellow), the same as R1 and R2. Glue the disc flat with epoxy, brass side down, and tape over it.

Calibrate it with real hits: record a few on the middle, the edge and the toe with the phone app, and compare the readings.

The 14-byte packet

What the bat sends 50 times a second, and how the dashboard reads it. The example was taken during a swing.

byte 0	byte 1	byte 2	byte 3	byte 4	byte 5	byte 6	byte 7	byte 8	byte 9	byte 10	byte 11	byte 12	byte 13
35	05	72	03	12	10	40	FD	06	0B	4C	01	00	00

ax	ay	az	gx	gy	gz	temperature
0x0535	0x0372	0x1012	0xFD40	0x0B06	0x014C	0x0000
= 1333 counts	= 882 counts	= 4114 counts	= -704 counts	= 2822 counts	= 332 counts	= 0 counts
/ 4096	/ 4096	/ 4096	/ 16.4	/ 16.4	/ 16.4	not used by the dashboard
+0.325 g	**+0.215 g**	**+1.004 g**	**-42.9 deg/s**	**+172.1 deg/s**	**+20.2 deg/s**	

What the dashboard does with these six numbers

Total rotation = square root of gx squared + gy squared + gz squared = 178.5 degrees per second

Bat speed = total rotation x pi / 180 x 0.85 m x 3.6 = 9.5 km/h

Impact = square root of ax squared + ay squared + az squared = 1.08 g

Shot direction = leg side if gz is above +1, off side if below -1, otherwise straight = leg side

Low byte first. The number 0x0B06 travels as 06 then 0B. Both ends must agree on this, and on the order of the seven numbers.

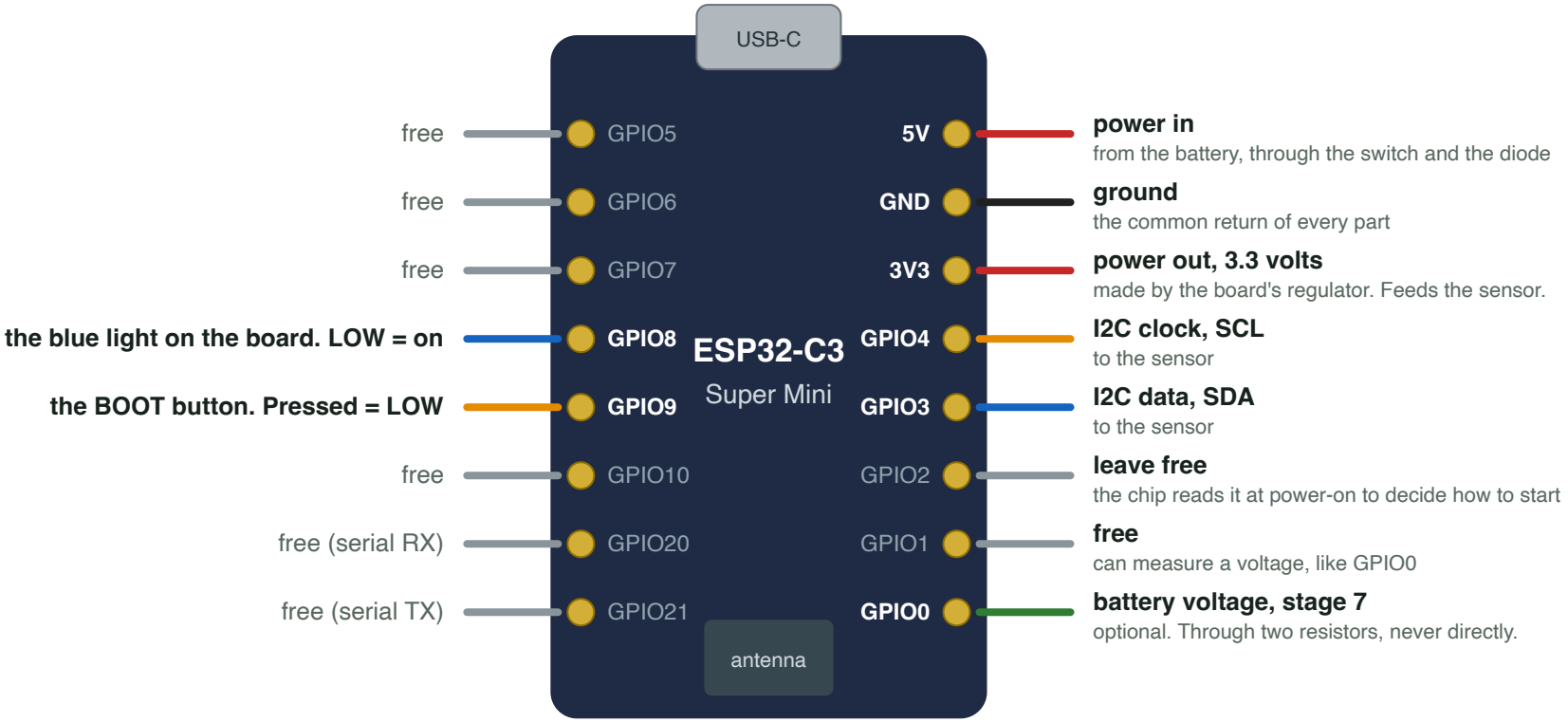
Signed. Each number runs from -32768 to +32767. A value with its top bit set is negative: 0xFD40 is -704.

Why counts and not units. Seven whole numbers fit in 14 bytes. The same values as text, as stage 9 sends them over WiFi, need about 110 bytes.

The scale, 4096 and 16.4, comes from the sensor's ranges of 8 g and 2000 degrees per second. Change a range and both ends have to change.

The ESP32-C3 Super Mini and what this kit uses each pin for

Seen from the top, USB socket up. The labels are printed on the underside of the real board.



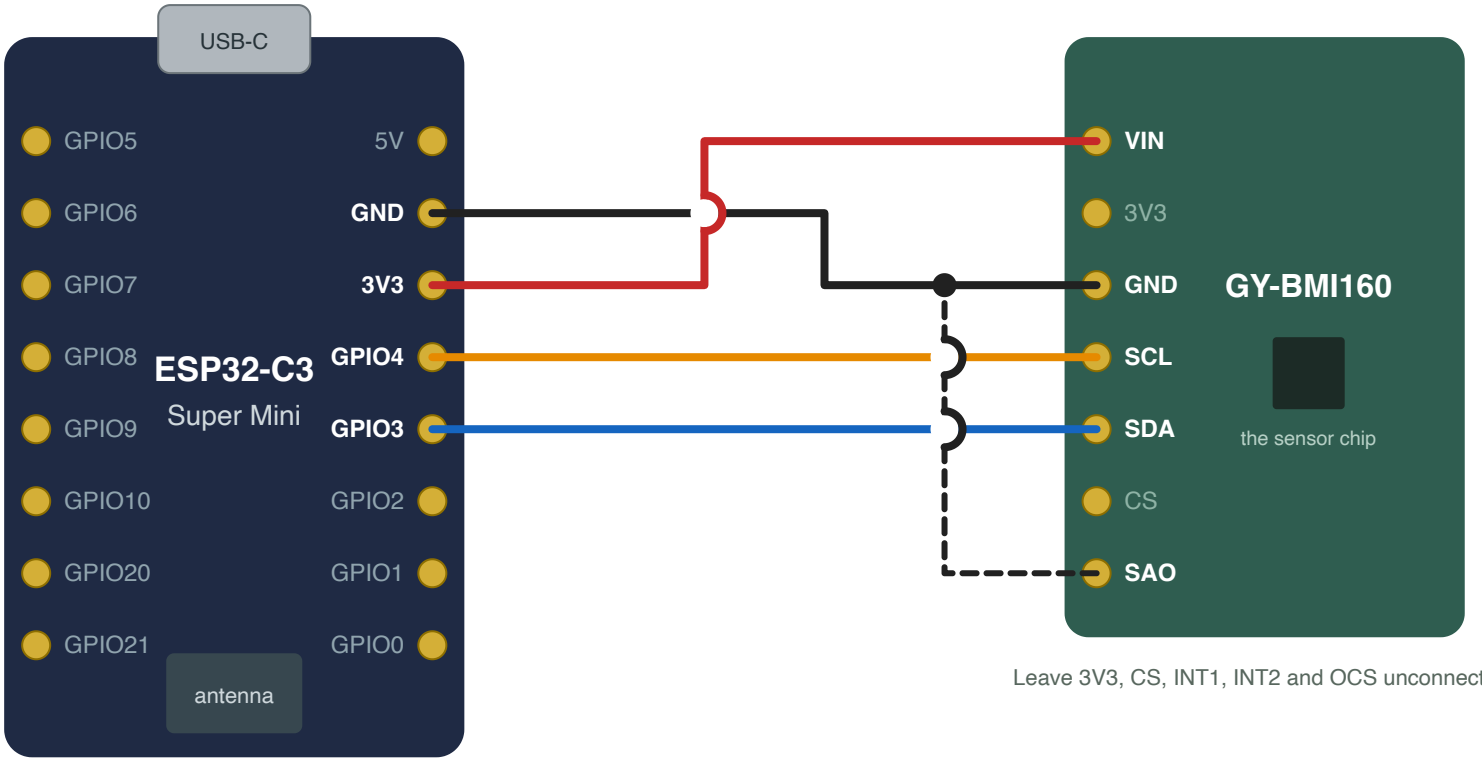
GPIO8 and GPIO9 are already used on the board itself, by the light and the button. Do not wire anything else to them.

GPIO2, GPIO8 and GPIO9 are read by the chip at the moment it gets power. A part that pulls one of them the wrong way stops it from starting.

Every pin works at 3.3 volts. Never connect 5 volts or the battery directly to a GPIO pin.

Stages 2 to 5: the sensor on five wires

Powered from the USB cable. Wire by the labels printed on your boards, not by their position in this drawing.



Leave 3V3, CS, INT1, INT2 and OCS unconnected.

The light is on GPIO8 and the BOOT button on GPIO9.

3V3 to VIN — power, 3.3 volts

GND to GND — ground

GPIO4 to SCL — I2C clock

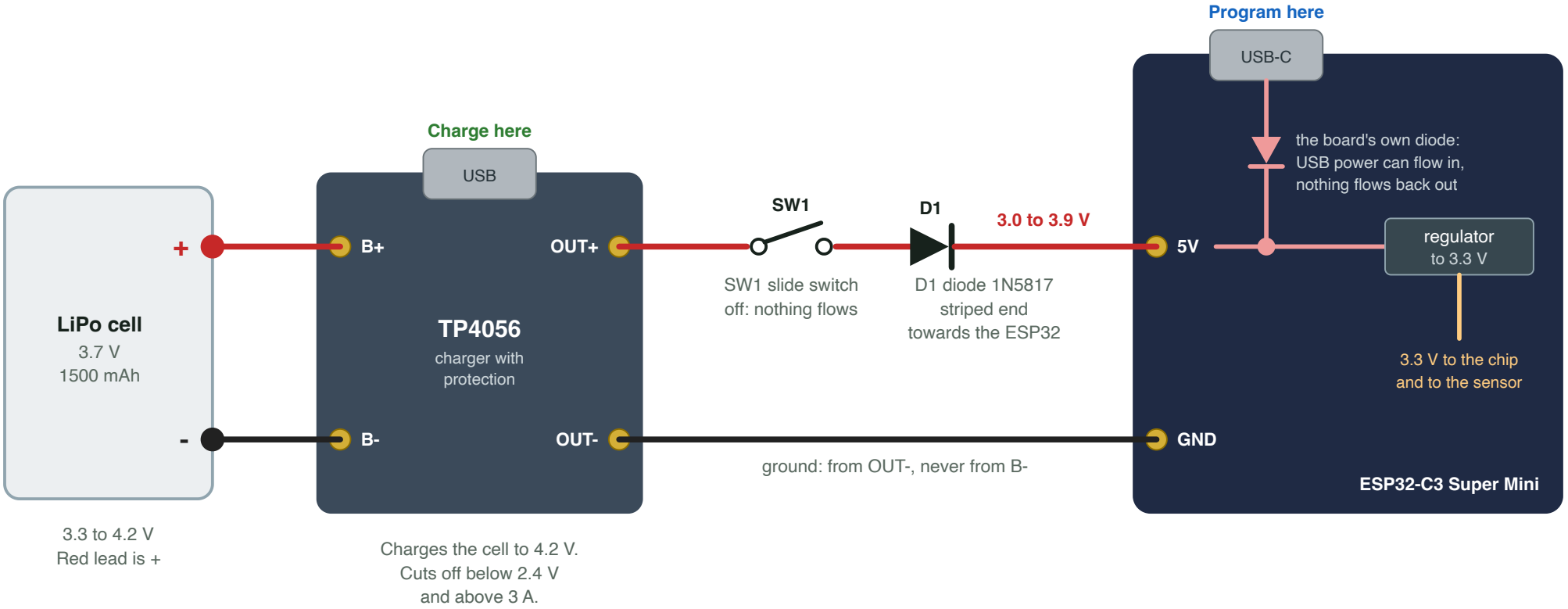
GPIO3 to SDA — I2C data

SAO to GND — fixes the sensor's address at 0x68. Left open it is usually 0x69, which also works.

Power the sensor from 3V3, never from 5V.

Stage 6: where the power flows

The cell is charged by the TP4056 and feeds the ESP32 through the switch and the diode.



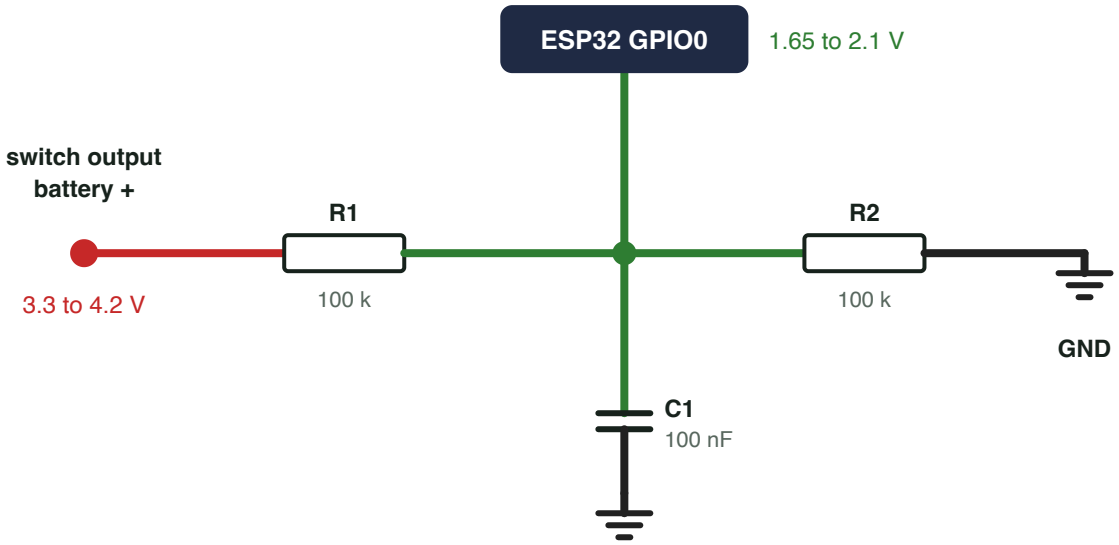
Why the diode. The ESP32 board joins its 5V pin to its USB socket through a diode of its own. Without D1, plugging in that USB cable would push 5 volts straight into the cell, with nothing to limit the current. D1 lets current flow out of the battery only.
With D1 fitted, the battery and the ESP32's USB cable can be connected at the same time. Whichever gives the higher voltage feeds the board.

Charging. Plug a phone charger into the TP4056's own socket, with the switch off. One light shows while it charges, the other when it is full.
Ground. B- is the cell's own minus. OUT- is the same point after the protection switch. Everything else connects to OUT-, so the protection can act.
Voltage. The diode costs about 0.3 V and the regulator needs about 0.1 V more than it gives. The board runs properly down to a cell voltage of about 3.5 V.

Without D1: never have the battery switched on and the ESP32's USB cable plugged in at the same time.

Stage 7 (optional): measuring the battery

Two equal resistors halve the battery voltage, so the pin never sees more than it can take.



How it works

The same small current flows through R1 and then R2. Because they are equal, each drops half of the battery voltage. The point between them is at half.

pin = battery x R2 / (R1 + R2)

4.2 V at the battery is 2.1 V at the pin. The pin measures up to about 2.5 V.

The firmware multiplies by 2 again. C1 holds the voltage steady for the moment the chip takes its measurement.

Take the battery voltage after the switch and before the diode. Then the resistors draw nothing while the bat is switched off.

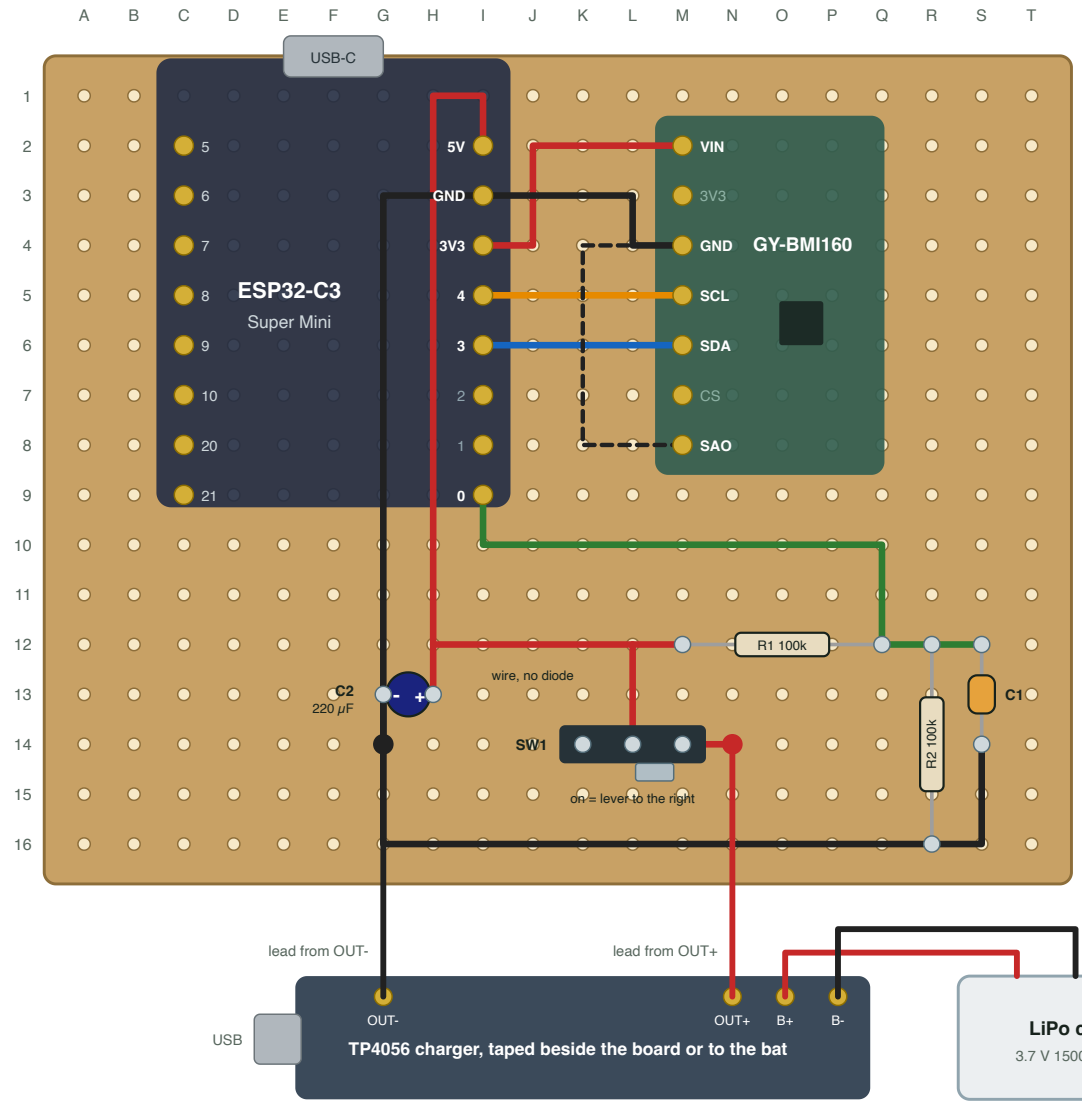
With 200 kilohm in total they draw 21 millionths of an amp while it is on, which the cell does not notice.

Resistors are never exactly equal. Stage 7 shows how to compare with a multimeter once and correct the difference.

Change the resistors and the firmware has to know: BATTERY_DIVIDER_RATIO in config.h is (R1 + R2) / R2.

Zero PCB layout, seen from the component side, without the diode

One arrangement that works. Each dot is a hole, 2.54 mm from the next. The wires run on the solder side and must be insulated.



Wires, pad to pad

I4 to M2	3V3 to VIN
I3 to M4	GND to GND
I5 to M5	GPIO4 to SCL
I6 to M6	GPIO3 to SDA
M8 to M4	SAO to GND
N14 to M14	charger OUT+ to the switch
L14 to L12	switch output
L12 to H12	a wire where the diode was
H12 to I2	on to 5V
G14 to I3	charger OUT- to GND
H13 to H12	C2 plus to the 5V line
G13 to G14	C2 minus to GND

Only for stage 7

L12 to M12	switch output to R1
Q12 to R12	R1 to R2
R12 to S12	R2 to C1
Q12 to I9	R1 and R2 to GPIO0
S14 to R16	C1 to the ground end of R2
R16 to G14	that ground end to OUT-

Parts

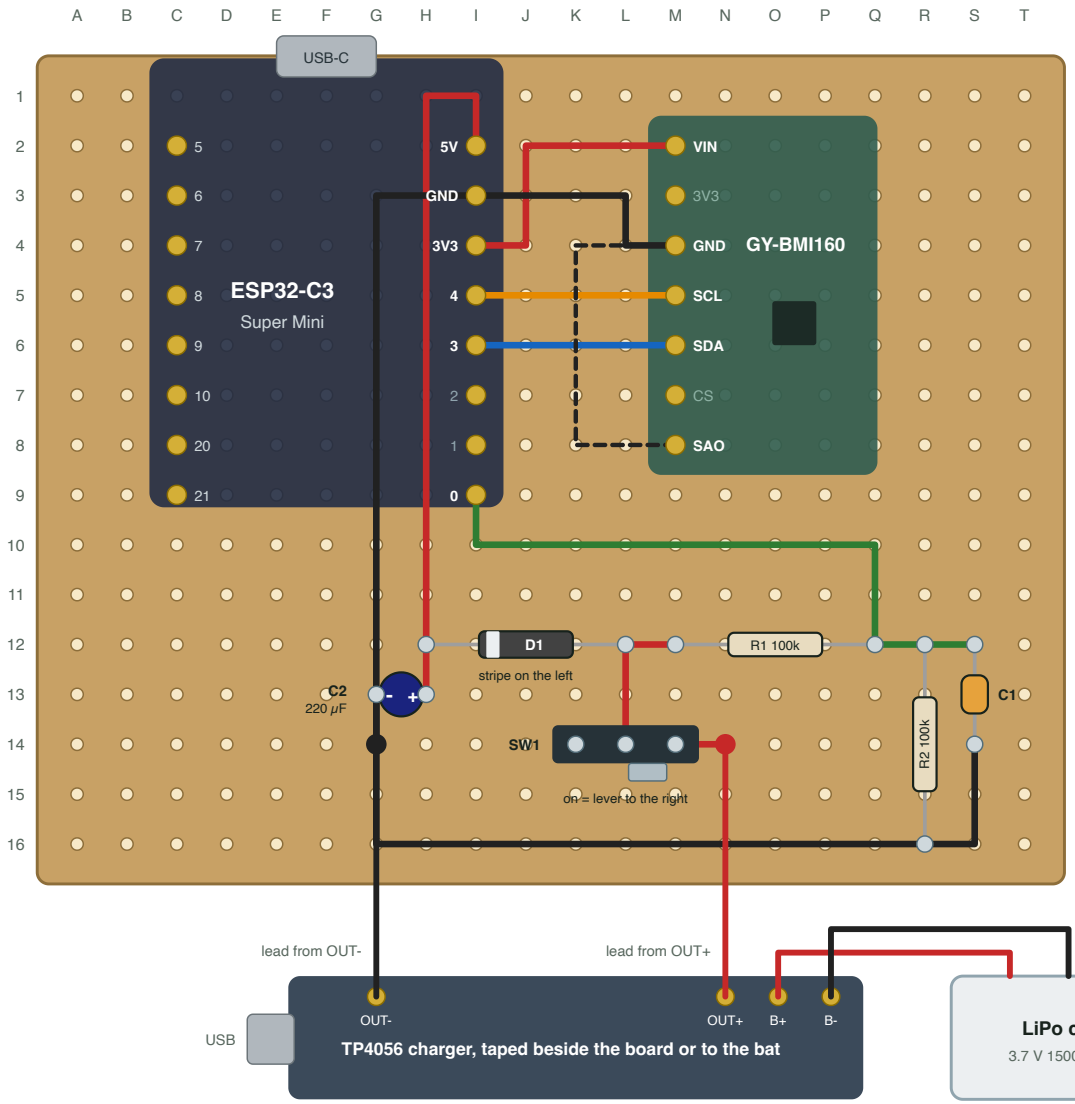
```
SW1 K14, L14, M14
C2 H13 (+) and G13 (−)
R1 M12 and Q12
R2 R12 and R16
C1 S12 and S14
OUT+ lead N14
OUT− lead G14
```

The hole positions assume this pin order: 5V, GND, 3V3, 4, 3, 2, 1, 0 down the right side of the ESP32, and VIN, 3V3, GND, SCL, SDA, CS, SAO on the sensor.
If your boards are printed in another order, keep the label-to-label connections of the table in the README and move the wires to suit.
The script that drew this checked it: the layout makes all 7 on-board connections of the circuit, joins nothing else, and no wire runs through another connection's pad.
Some wires cross each other, which is why they must be insulated. The two leads from the charger are soldered at N14 and G14 and leave the board there.

Without the diode: switch the battery off before plugging a USB cable into the ESP32, and on again after unplugging it.

Zero PCB layout, seen from the component side

One arrangement that works. Each dot is a hole, 2.54 mm from the next. The wires run on the solder side and must be insulated.



Wires, pad to pad

I4 to M2	3V3 to VIN
I3 to M4	GND to GND
I5 to M5	GPIO4 to SCL
I6 to M6	GPIO3 to SDA
M8 to M4	SAO to GND
N14 to M14	charger OUT+ to the switch
L14 to L12	switch to the diode
H12 to I2	diode to 5V
G14 to I3	charger OUT- to GND
H13 to H12	C2 plus to the 5V line
G13 to G14	C2 minus to GND

Only for stage 7

L12 to M12	switch output to R1
Q12 to R12	R1 to R2
R12 to S12	R2 to C1
Q12 to I9	R1 and R2 to GPIO0
S14 to R16	C1 to the ground end of R2
R16 to G14	that ground end to OUT-

Parts

```
D1 H12 (stripe) and L12
SW1 K14, L14, M14
C2 H13 (+) and G13 (−)
R1 M12 and Q12
R2 R12 and R16
C1 S12 and S14
OUT+ lead N14
OUT− lead G14
```

The hole positions assume this pin order: 5V, GND, 3V3, 4, 3, 2, 1, 0 down the right side of the ESP32, and VIN, 3V3, GND, SCL, SDA, CS, SAO on the sensor.
If your boards are printed in another order, keep the label-to-label connections of the table in the README and move the wires to suit.
The script that drew this checked it: the layout makes all 8 on-board connections of the circuit, joins nothing else, and no wire runs through another connection's pad.
Some wires cross each other, which is why they must be insulated. The two leads from the charger are soldered at N14 and G14 and leave the board there.